

Universidad Centroamericana

José Simeón Cañas



Bases de datos

Sistema de gestion clinica de veterinaria

Estudiantes:

- Melara Munguia Diego Leonardo 00180825
- Colocho Andrade Robert Stanley 00054625

Fecha de entrega: viernes 19 de junio de 2026

Descripción general del escenario:

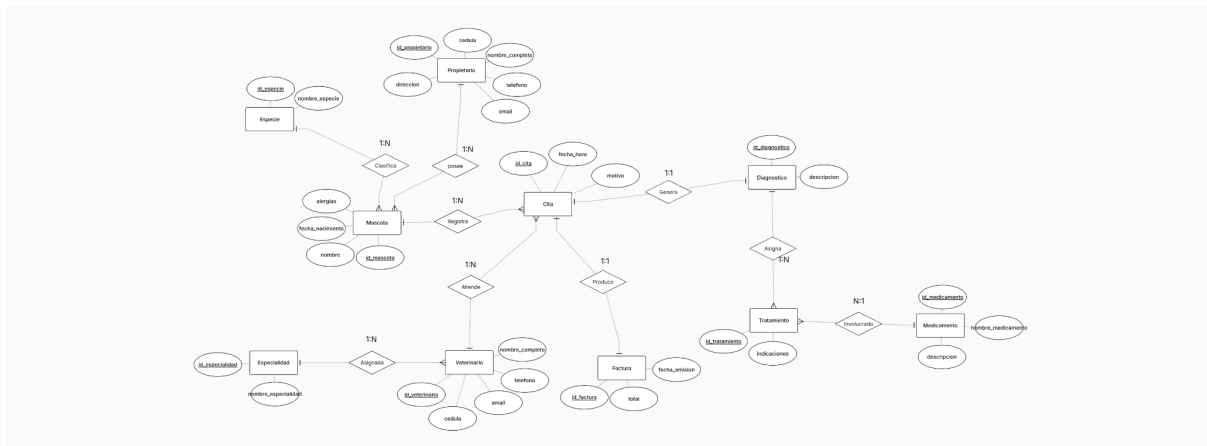
El presente proyecto implementa un Sistema de Gestión Clínica Veterinaria completo en PostgreSQL. El sistema permite gestionar propietarios de mascotas, mascotas, veterinarios, citas, diagnósticos, tratamientos, medicamentos y facturación.

Escenario seleccionado: Escenario A - Sistema de Gestión Clínica Veterinaria

Reglas de negocio implementadas:

- Alergia a medicamentos: Un propietario puede tener varias mascotas
- Especialidad veterinaria: Cada mascota pertenece a una sola especie
- Citas con diagnóstico: Una cita es atendida por un solo veterinario
- Único diagnóstico por cita: Cada cita genera un solo diagnóstico
- Tratamiento multimedicamentoso: Un diagnóstico puede tener múltiples tratamientos
- Facturación automática: Se genera factura al registrar diagnóstico
- Validación de alergias: Prevent prescripción de medicamentos alérgicos
- Validación de fechas: No se permiten citas en fechas pasadas

Diagrama Entidad-Relación:



Esquema relacional normalizado (3FN)



Diccionario de datos:

Tabla: Especie

Campo	Tipo	Restricciones	Descripcion
id_especie	serial	pk	identificador unico
nombre_especie	varchar(50)	unique, not null	nombre de la especie

Tabla: Especialidad

Campo	Tipo	Restricciones	Descripcion
id_especialidad	serial	pk	identificador unico
nombre_especialidad	varchar(50)	unique, not null	nombre especialidad

Tabla: Medicamento

Campo	Tipo	Restricciones	Descripcion
id_medicamento	serial	pk	identificador unico
nombre_medicamento	varchar(100)	not null	nombre del medicamento
descripcion	text	-	descripcion del uso

Tabla: Propietario

Campo	Tipo	Restricciones	Descripcion
id_propietario	serial	pk	identificador unico
cedula	varchar(20)	unique, not null	cedula unica
nombre_completo	varchar(100)	not null	nombre completo
telefono	varchar(20)	-	numero de contacto

email	varchar(100)	-	correo electronico
direccion	text	-	direccion de residencia

Tabla: Veterinario

Campo	Tipo	Restricciones	Descripcion
id_veterinario	serial	pk	identificador unico
cedula	varchar(20)	unique, not null	cedula unica
nombre_completo	varchar(100)	not null	nombre completo
telefono	varchar(20)	-	numero de contacto
email	varchar(100)	-	correo electronico
id_especialidad	int	fk, not null	referencia a especialidad

Tabla: Mascota

Campo	Tipo	Restricciones	Descripcion
id_mascota	serial	pk	identificador unico
nombre	varchar(50)	not null	nombre de la mascota
fecha_nacimiento	date	-	fecha de nacimiento
id_propietario	int	fk, not null	referencia a propietario
id_especie	int	fk, not null	referencia a especie
alergias	text	-	alergias conocidas

Tabla: Cita

Campo	Tipo	Restricciones	Descripcion
id_cita	serial	pk	identificador unico
fecha_hora	timestamp	not null	fecha y hora de la consulta
motivo	varchar(200)	-	motivo de consulta
id_mascota	int	fk, not null	referencia a mascota
id_veterinario	int	fk, not null	referencia a veterinario

Tabla: Diagnóstico

Campo	Tipo	Restricciones	Descripcion
id_diagnostico	serial	pk	identificador unico
descripcion	text	not null	descripcion del diagnostico
id_cita	int	fk, not null, unique	referencia a cita

Tabla: Tratamiento

Campo	Tipo	Restricciones	Descripcion
id_tratamiento	serial	pk	identificador unico
indicaciones	text	-	instrucciones de uso
id_diagnostico	int	fk, not null	referencia a diagnostico
id_medicamento	int	fk	referencia a medicamento

Tabla: Factura

Campo	Tipo	Restricciones	Descripcion
id_factura	serial	pk	identificador unico
fecha_emision	timestamp	default current_timestamp	fecha de emision
total	decimal(10, 2)	check>=0, not null	monto total
id_cita	int	fk, unique, not null	referencia a cita

Consultas sql principales:

Consulta 1: Citas por mes:

```
select
    extract(month from c.fecha_hora) as mes,
    m.nombre as mascota,
    count(*) as total_citas
from Cita c
join Mascota m on c.id_mascota = m.id_mascota
where extract(year from c.fecha_hora) = 2025
group by mes, m.nombre
order by mes, total_citas desc;
```

Analiza el volumen de citas por mascota en cada mes

Consulta 2: Veterinario más activo:

```
select
    v.nombre_completo as veterinario,
    e.nombre_especialidad,
    count(*) as total_citas
from Cita c
join Veterinario v on c.id_veterinario = v.id_veterinario
join Especialidad e on v.id_especialidad = e.id_especialidad
where c.fecha_hora between '2025-01-01' and '2025-03-31'
group by v.id_veterinario, v.nombre_completo, e.nombre_especialidad
order by total_citas desc
limit 1;
```

Identifica al veterinario con mayor carga de trabajo en un período

Consulta 3: Medicamentos más utilizados:

```
select
    m.nombre_medicamento,
    m.descripcion,
    count(t.id_tratamiento) as veces_prescrito
from Medicamento m
left join Tratamiento t on m.id_medicamento = t.id_medicamento
group by m.id_medicamento, m.nombre_medicamento, m.descripcion
order by veces_prescrito desc;
```

Ranking de medicamentos para control de inventario

Consulta 4: Ingresos por especialidad:

```
select
    e.nombre_especialidad,
    count(f.id_factura) as numero_facturas,
    sum(f.total) as ingresos_totales,
    round(avg(f.total), 2) as promedio_por_factura
from Factura f
join Cita c on f.id_cita = c.id_cita
join Veterinario v on c.id_veterinario = v.id_veterinario
join Especialidad e on v.id_especialidad = e.id_especialidad
group by e.id_especialidad, e.nombre_especialidad
order by ingresos_totales desc;
```

Análisis de rentabilidad por área de especialización

Consulta 5: Seguimiento de clientes:

```
select distinct p.nombre_completo as propietario, p.telefono, p.email
from Propietario p
join Mascota m on p.id_propietario = m.id_propietario
left join Cita c on m.id_mascota = c.id_mascota
    and c.fecha_hora >= current_date - interval '6 months'
where c.id_cita is null;
```

Identifica propietarios con mascotas sin citas recientes

Consulta 6: Alergias registradas:

```
select
    m.nombre as mascota,
    es.nombre_especie,
    p.nombre_completo as propietario,
    m.alergias
from Mascota m
join Especie es on m.id_especie = es.id_especie
join Propietario p on m.id_propietario = p.id_propietario
where m.alergias is not null and m.alergias != 'Ninguna'
order by m.nombre;
```

Consulta de seguridad para medicamentos con riesgo

Consulta 7: Diagnósticos y estadísticas

```
select
    d.descripcion as diagnostico,
    count(d.id_diagnostico) as numero_casos,
    round(avg(f.total), 2) as costo_promedio_tratamiento
from Diagnostico d
left join Tratamiento t on d.id_diagnostico = t.id_diagnostico
left join Factura f on d.id_cita = f.id_cita
group by d.descripcion
order by numero_casos desc;
```

Análisis epidemiológico de padecimientos tratados

Documentación de las funciones:

1. historial_clinico(p_id_mascota)

Propósito: Obtener el historial clínico completo de una mascota Parámetros: ID de la mascota Retorna:

fecha: Fecha de la cita

motivo: Motivo de la consulta

veterinario: Nombre del veterinario

diagnostico: Descripción del diagnóstico

medicamentos: Listado de medicamentos prescritos

total: Total facturado

Uso:

```
SELECT * FROM historial_clinico(1);
```

2. verificar_alergias()

Propósito: Trigger que valida medicamentos contra alergias de mascota

Comportamiento: Se ejecuta AFTER INSERT en tabla Diagnóstico Lógica:

Obtiene las alergias de la mascota

Verifica medicamentos prescritos contra las alergias

Lanza excepción si hay conflicto

Impide registro del diagnóstico con medicamento alérgico

Procedimientos almacenados:

1. registrar_propietario

Propósito: Registrar un nuevo propietario en el sistema Parámetros:

p_cedula: Cédula del propietario

p_nombre: Nombre completo

p_telefono: Número de teléfono

p_email: Correo electrónico

p_direccion: Dirección de residencia

p_id_propietario (OUT): ID generado

Validaciones: Verifica que la cédula sea única


```

create or replace procedure registrar_propietario(
    p_cedula varchar,
    p_nombre varchar,
    p_telefono varchar,
    p_email varchar,
    p_direccion text,
    out p_id_propietario int
) as $$
begin
    insert into Propietario (cedula, nombre_completo, telefono, email, direccion)
    values (p_cedula, p_nombre, p_telefono, p_email, p_direccion)
    returning id_propietario into p_id_propietario;

    raise notice 'Propietario registrado exitosamente con ID: %', p_id_propietario;
exception when unique_violation then
    raise exception 'Error: La cedula % ya está registrada', p_cedula;
end;
$$ language plpgsql;

```

2. registrar_mascota

Propósito: Registrar una nueva mascota en el sistema Parámetros:

p_nombre: Nombre de la mascota

p_fecha_nacimiento: Fecha de nacimiento

p_id_propietario: ID del propietario dueño

p_id_especie: ID de la especie

p_alergias: Alergias conocidas

p_id_mascota (OUT): ID generado

Validaciones: Verifica existencia de propietario y especie

```

create or replace procedure registrar_mascota(
    p_nombre varchar,
    p_fecha_nacimiento date,
    p_id_propietario int,
    p_id_especie int,
    p_alergias text,
    out p_id_mascota int
) as $$
begin
    -- Validar que el propietario exista
    if not exists (select 1 from Propietario where id_propietario = p_id_propietario) then
        raise exception 'Error: El propietario con ID % no existe', p_id_propietario;
    end if;

    -- Validar que la especie exista
    if not exists (select 1 from Especie where id_especie = p_id_especie) then
        raise exception 'Error: La especie con ID % no existe', p_id_especie;
    end if;

    insert into Mascota (nombre, fecha_nacimiento, id_propietario, id_especie, alergias)
    values (p_nombre, p_fecha_nacimiento, p_id_propietario, p_id_especie, coalesce(p_alergias, 'Ninguna'))
    returning id_mascota into p_id_mascota;

    raise notice 'Mascota registrada exitosamente con ID: %', p_id_mascota;
end;
$$ language plpgsql;

```

3. programar_cita

Propósito: Programar una nueva cita veterinaria Parámetros:

p_fecha_hora: Fecha y hora de la cita

p_motivo: Motivo de consulta

p_id_mascota: ID de la mascota

p_id_veterinario: ID del veterinario

p_id_cita (OUT): ID generado

Validaciones:

Verifica existencia de mascota y veterinario

Valida que no sea en fecha pasada

```
-- Verificar ID de la cita programada
create or replace procedure programar_cita(
    p_fecha_hora timestamp,
    p_motivo varchar,
    p_id_mascota int,
    p_id_veterinario int,
    out p_id_cita int
) as $$
begin
    -- Validar que la mascota exista
    if not exists (select 1 from Mascota where id_mascota = p_id_mascota) then
        raise exception 'Error: La mascota con ID % no existe', p_id_mascota;
    end if;

    -- Validar que el veterinario exista
    if not exists (select 1 from Veterinario where id_veterinario = p_id_veterinario) then
        raise exception 'Error: El veterinario con ID % no existe', p_id_veterinario;
    end if;

    -- Validar que la fecha no sea en el pasado
    if p_fecha_hora < current_timestamp then
        raise exception 'Error: No se pueden programar citas en fechas pasadas';
    end if;

    insert into Cita (fecha_hora, motivo, id_mascota, id_veterinario)
    values (p_fecha_hora, p_motivo, p_id_mascota, p_id_veterinario)
    returning id_cita into p_id_cita;

    raise notice 'Cita programada exitosamente con ID: %', p_id_cita;
end;
$$ language plpgsql;
```

4. registrar_diagnostico_y_factura

Propósito: Registrar diagnóstico y generar factura automáticamente Parámetros:

p_descripcion: Descripción del diagnóstico

p_id_cita: ID de la cita

p_total: Total a facturar

p_id_diagnostico (OUT): ID generado

Validaciones:

Verifica que la cita exista

Valida que no exista diagnóstico previo

Valida que el total sea positivo

```

create or replace procedure registrar_diagnostico_y_factura(
    p_descripcion text,
    p_id_cita int,
    p_total decimal,
    out p_id_diagnostico int
) as $$
begin
    -- Validar que la cita exista
    if not exists (select 1 from Cita where id_cita = p_id_cita) then
        raise exception 'Error: La cita con ID % no existe', p_id_cita;
    end if;

    -- Validar que no exista diagnóstico previo para esta cita
    if exists (select 1 from Diagnostico where id_cita = p_id_cita) then
        raise exception 'Error: Ya existe un diagnóstico registrado para esta cita';
    end if;

    -- Validar que el total sea positivo
    if p_total <= 0 then
        raise exception 'Error: El total debe ser mayor a 0';
    end if;

    -- Insertar diagnóstico
    insert into Diagnostico (descripcion, id_cita)
    values (p_descripcion, p_id_cita)
    returning id_diagnostico into p_id_diagnostico;

    -- Insertar factura automáticamente
    insert into Factura (total, id_cita)
    values (p_total, p_id_cita);

    raise notice 'Diagnóstico registrado con ID: % y factura generada', p_id_diagnostico;
end;
$$ language plpgsql;

```

5. actualizar_alergias_mascota

Propósito: Actualizar registro de alergias de una mascota Parámetros:

p_id_mascota: ID de la mascota

p_nuevas_alergias: Nuevas alergias a registrar

Validaciones: Verifica que la mascota exista

```

create or replace procedure actualizar_alergias_mascota(
    p_id_mascota int,
    p_nuevas_alergias text
) as $$
begin
    -- Validar que la mascota exista
    if not exists (select 1 from Mascota where id_mascota = p_id_mascota) then
        raise exception 'Error: La mascota con ID % no existe', p_id_mascota;
    end if;

    update Mascota
    set alergias = p_nuevas_alergias
    where id_mascota = p_id_mascota;

    raise notice 'Alergias de la mascota ID % actualizadas a: %', p_id_mascota, p_nuevas_alergias;
end;
$$ language plpgsql;

```

6. veterinario_mas_citas

Propósito: Identificar el veterinario con más citas en un período Parámetros:

p_fecha_inicio: Fecha inicio del período

p_fecha_fin: Fecha fin del período

Retorna: Nombre, cédula y cantidad de citas del veterinario más activo

```

create or replace procedure veterinario_mas_citas(
    p_fecha_inicio date,
    p_fecha_fin date
) as $$
begin
    select
        v.nombre_completo,
        v.cedula,
        count(c.id_cita) as total_citas
    from Veterinario v
    left join Cita c on v.id_veterinario = c.id_veterinario
        and date(c.fecha_hora) between p_fecha_inicio and p_fecha_fin
    group by v.id_veterinario, v.nombre_completo, v.cedula
    order by total_citas desc
    limit 1;
end;
$$ language plpgsql;

```

Disparadores creados:

trigger_verificar_alergias

Tabla: Diagnóstico

Evento: AFTER INSERT

Función: verificar_alergias()

Propósito: Validar seguridad al prescribir medicamentos Regla de negocio

implementada: "Al registrar un diagnóstico, verificar automáticamente si existen alergias registradas de la mascota a algún medicamento del tratamiento"

```

create trigger trigger_verificar_alergias
after insert on Diagnostico
for each row
execute function verificar_alergias();

```